

HerAgent: A Block-Level Harness for Coding-Agent Environment Setup

1st Anon.

authors omitted for anonymous review

Abstract—This document is a model and instructions for `LaTeX`. This and the `IEEEtran.cls` file define the components of your paper [title, text, heads, etc.]. ***CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

Index Terms—component, formatting, style, styling, insert.

I. INTRODUCTION

Modern software engineering agents are increasingly deployed and expected to operate on real repositories: running tests, reproducing bugs, applying patches, launching services, and validating changes. Automated environment setup is therefore a prerequisite for making AI coding agents practically useful. A repository-level task becomes executable only after the agent has reconstructed the required packages, runtimes, dependencies, and build assumptions. In practice, this process is long-horizon, stateful, and failure-prone.

Existing work has made important progress on coding-agent environment setup [1]–[7]. At a high level, these systems combine an LLM-driven controller with an execution and testing harness. The agent inspects and reasons about the repository, and selects which commands to run. The harness executes the commands in a sandbox and provides the results back to the agent, overall managing progress toward a usable environment. The output of such systems is typically a trajectory of commands to execute, or a generated setup artifact like a Bash script or a Dockerfile.

Long or complicated setup trajectories typically involve an agent validating partial progress and recovering from invalid states produced when attempted actions fail. A major reason both validation and recovery are difficult is that the correctness signal provided by an individual setup or install step is typically extremely weak relative to the overall success criterion. For example, one important step in environmental configuration is installing necessary package dependencies. However, a package manager can locally report whether a package installed correctly or not, not whether the package is the correct version and does not conflict with other packages installed at different steps (before or after). The global check entails evaluating whether the overall project can be built and run, which can only happen when the process is complete or nearly-complete. This additionally means, as an implication, that when a step *does* fail, the command that actually introduced the problem is very often due to a prior step (up to **FIXME: X%** of commands in our experiments).

This means that, beyond iteratively selecting the next best command to try over an environmental setup task, setup agents also typically track *intermediate environment states* to support failure recovery (rollback) and retry.

Our key insight is that, at least implicitly, environment setup proceeds through relatively discrete functional stages such as installing system dependencies, provisioning a language runtime, or resolving project-specific dependencies. Each stage functions to group dependent or related commands into a cohesive unit, often with a natural validation boundary. As a result, environmental setup harnesses should track and recover intermediate states at the *same granularity* as the functional structure of the overall setup process.

Prior work varies in the granularity at which it maintains and reasons about intermediate setup state. Single-command harnesses [2], [3] work one command at a time. Some of these approaches checkpoint after each command to enable rollback (e.g., Repo2Run, using committed Docker snapshots [3]; rollback improves runnability by 3.1%). However, the fineness of command-level granularity has drawbacks. First, querying a model for every command is expensive (see Table ??). Second, command-level rollback targets the command that just failed, regardless of rollback point. Repo2Run, for example, either reverts a command to the last committed snapshot, or resets the container and starts over [3]. Third, a command-level trajectory mixes correct commands, failed trials, and temporary fixes together with implicit shell state, so a clean and reusable artifact is hard to extract from it afterward [3]. Neither option handles the common case of commands that fail for difficult-to-localize prior reasons (instead reasoning only about the failed command). At the opposite extreme, whole-script harnesses [4], [7] generate an entire Dockerfile or Bash script and treat the whole file as the unit of progress without intermediate checkpoints. For example, Installmatic repairs a failed Dockerfile by rebuilding it from a fresh agent that has no memory of earlier progress, with repair capped at two attempts. It reports installing 21 of 40 repositories (55%) [7].

In this paper, we propose **HerAgent** (Harness **Envi**Ronment Setup Agent), a coding agent environment setup system that maintains setup progress as an ordered sequence of block-level `BashFiles` and sandbox checkpoints. A *block* groups the dependent commands of one functional stage, such as installing system dependencies or provisioning a language runtime. A block is the basic unit for planning, validation, checkpointing, and recovery; it is intended to be small enough to validate and repair, but large enough to preserve meaningful

progress checkpoints We call this mechanism a *Block-Level Checkpoint*. Each block is represented as a `BashFile` and corresponds to a meaningful setup stage.

Upon inspecting initial repository artifacts to understand the project context, HerAgent synthesizes a setup plan in a one-shot manner, automatically partitioning the expected required commands into distinct, semantically meaningful functional blocks. Each block is assigned specific validation targets. HerAgent advances block by block with per-block validation. When a block fails, HerAgent first *localizes* the block that introduced the fault, which is not always the block that failed. If the failing block is itself the cause, HerAgent repairs it in place at the command level. If an earlier block is the cause, the blocks that ran after it have built on a faulty state, so HerAgent rolls back to the checkpoint before that block and repairs it there.

We evaluate HerAgent on three environment-setup benchmarks. These are the ExecutionAgent dataset, SetupBench, and Repo2Run. To our knowledge, HerAgent is the first harness to track and recover coding-agent setup state at the granularity of functional blocks. It is designed to reduce wasted exploration, preserve validated progress, and produce environments that can be reconstructed from a clean state. We summarize our contributions as follows:

- We formulate automated coding-agent environment setup as a progress-control harness problem, and identify the granularity limitation.
- We introduce Block-Level Checkpoint. Based on this, we design and implement HerAgent, which combines block-level execution and validation, local repair, and checkpoint-based rollback.
- We evaluate HerAgent on ExecutionAgent, SetupBench, and Repo2Run, comparing against prior environment-setup agents and analyzing effectiveness, reproducibility, efficiency, and progress-control behavior.
- We release our implementation, benchmark subset, validation commands, and experiment artifacts to support reproducible follow-up research.

II. MOTIVATION

CLG: this is too wordy, but is a better approach here, I think Figure 1 shows a motivating example based on the setup of DataHub. DataHub is an open-source metadata platform for data discovery and governance, first developed at LinkedIn and now maintained as a community project under the Apache 2.0 license.¹ The single repository spans several language ecosystems. The metadata service is a Java application built with Gradle, its web interface is a React frontend served through Node and Yarn, and it ships Python tooling for ingestion.

Figure 1 shows an illustrative attempt to automatically configure this environment. In the first attempt, an agent may successfully install system and language dependencies (❶, commands 1–3), bringing up the local DataHub stack

¹<https://github.com/datahub-project/datahub>; setup commands discussed in the developer guide at <https://docs.datahub.com/docs/developers>.

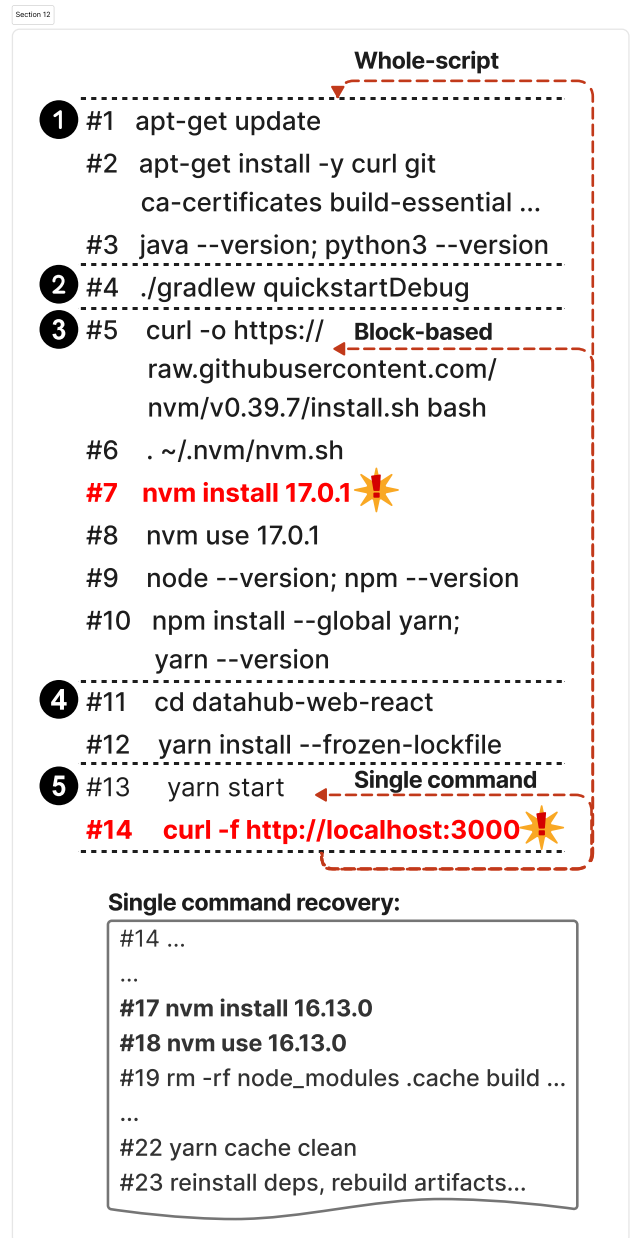


Fig. 1. A portion of a command trajectory in configuring DataHub. In the first attempt, the successful installation of the wrong version of Node (#7) leads to a failure at #14. A whole-script harness attempts to regenerate the script from scratch, redoing straightforward setup. A single-command harness corrects from the failed command, entailing cleanup of the polluted environment. Our block-based harness infers the required setup stages as *blocks* that group related commands, reasons about a failure to localize the block that caused it, and rolls back to the checkpoint before it. **CLG: OK it's obviously messy (it's in figma, to be fixed), but...single column??? I THINK this conveys all the necessary info. My thinking was to give the block labels in the text.**

(❷, command 4), provision a Node and Yarn runtime (❸, commands 5–10), and then install the frontend dependencies (❹, commands 11–12). The version checks all pass, and dependencies all appear to install successfully. However, when attempting to start the frontend and confirm that it is serving (❺), command 14 will fail, because of an OpenSSL/WebPack

error. The runtime chosen and installed at commands 7–8, Node 17, ships with OpenSSL 3, which removed the legacy hashing that the version of WebPack used in the frontend relies on.

This type of failure—where the command that fails is not the actual root cause of the failure—is a characteristic challenge in environmental setup. Existing approaches therefore typically provide a recovery and retry mechanism, illustrated in Figure 1. Some, which we refer to as *whole-script* harnesses (e.g., PiPer [?], Installamatic [?]) attempt to regenerate the build file or script from scratch (“whole-script” in Figure 1). This resets the environment and produces a clean command trajectory if ultimately successful, but wastes computational and LLM consumption repeating straightforward setup steps unrelated to the error (like the boilerplate `apt` installation of basic packages and corresponding validation checks in commands 1–3).

Other techniques begin recovery reasoning from the failed command, with a goal of undoing potentially-polluted environmental state before trying again. Repo2Run, for example, snapshots the environment per-command and, when a command fails, reverts to the state just before it [3]. While this may undo any change in state caused by the failed command (none, here, for command 13), it often (as here) does not address the root cause. Recovery from the incorrect Node choice entails significant cleanup of the polluted state. The “Single command” line shows the recovery back to the previous command’s snapshot, and “Single command recovery” block in Figure 1 shows a portion of the recovery trajectory required (an additional 9–10 commands, in this case, not all shown).

The “Block-based” line in Figure 1 corresponds to the novel control and recovery mechanism we present in this paper, reified in HerAgent. HerAgent begins from the repository and a target commit and inspects the project artifacts, such as build files, lockfiles, and setup documentation. It does this to infer a plan, or a partition, of the required environmental setup steps into an ordered sequence of *blocks*. For this HerAgent run on DataHub, the plan entailed five blocks, corresponding to the numbered segments in the Figure. Her agent then generates and validates setup commands by proceeding sequentially through the required blocks.

Block-based reasoning affords several advantages. First, by proceeding forwards in a block-wise fashion, HerAgent can reason about multiple related commands at once, rather than requiring an LLM-based decision for each one. For example, once the agent has determined that the project needs Java and Python, the system-package installs follow from the decision and do not each require individual reasoning.

Second, blocks provide natural validation barriers as well as checkpointing and decision boundaries. Checks that confirm successful installation, such as `java -version` and `python3 --version`, provide evidence that a stage succeeded. These checks correspond to natural block boundaries that provide meaningful validation of stage completion, and natural moments to record checkpoints, without requiring a snapshot or decision after every routine command.

A natural benefit of staged checkpointing is more effective failure recovery, especially in light of non-local root causes (like in this DataHub configuration problem). When a block fails its validation (like in block 5), HerAgent reasons about the mistake to localize it to the block that *introduced* the fault, which may not be the same as the block that failed (block 3, here, where the runtime was selected and installed). The blocks that ran after the fault-inducing block built on polluted state. HerAgent thus returns to the checkpoint before it and attempts to recover from there.

In this example, this allows HerAgent to rollback to the checkpoint before the runtime is installed, install the correct one, and then replay Blocks 3–5 accordingly. This leaves the validated and correct prefix from blocks 1–2 unchanged (without repeating work), avoids the teardown and cleanup necessitated by single command reasoning, and ultimately produces a trajectory that installs the environment from a clean state in order and without interleaved failure and cleanup attempts.

We describe the HerAgent approach in detail in the next Section.

III. APPROACH

CLG: I copied this paragraph from the intro when I went through it; I like the way it decomposes the problem/frames it and it maps onto HerAgent’s phases well. Might be worth reusing/integrating, so for now I’m just moving it around to save it. We view this as a *progress harness problem*. An environment-agent harness manages setup execution inside a sandbox and turns a successful setup trajectory into a reproducible artifact. The harness need to (1) decide how to advance forward from an empty sandbox toward usable states; (2) decide where to place validation boundaries that make partial progress trustworthy; and (3) decide where to place recovery boundaries so that failures do not pollute later setup steps. These decisions make progress granularity a central design choice. CLG: end of copied text

This section describes the complete harness workflow, the block-state representation used to control progress, and the repair and replay loop used after failures. The input of HerAgent is a repository URL and its tag or commit, an initial Dockerfile, based on which an isolated Docker container is constructed and the target repository is cloned in. The output is a reproducible setup package, which includes a sequence of block-structured executable Bash files, the final validated docker container, and a final validation report.

The harness controls of HerAgent include four phases, which include the blocks generation and management, and legal state transitions, such as execution, validation, checkpointing, repairing, rollback, and replay: (1)Repository Inspection, (2)Block-State Construction and Forward Progress, (3)Failure Recovery through Repair and Rollback, and (4)Final Clean Replay.

A. Repository Inspection

CLG: One of the other common things I say on my list/decision tree is: “there’s lots of what but not enough

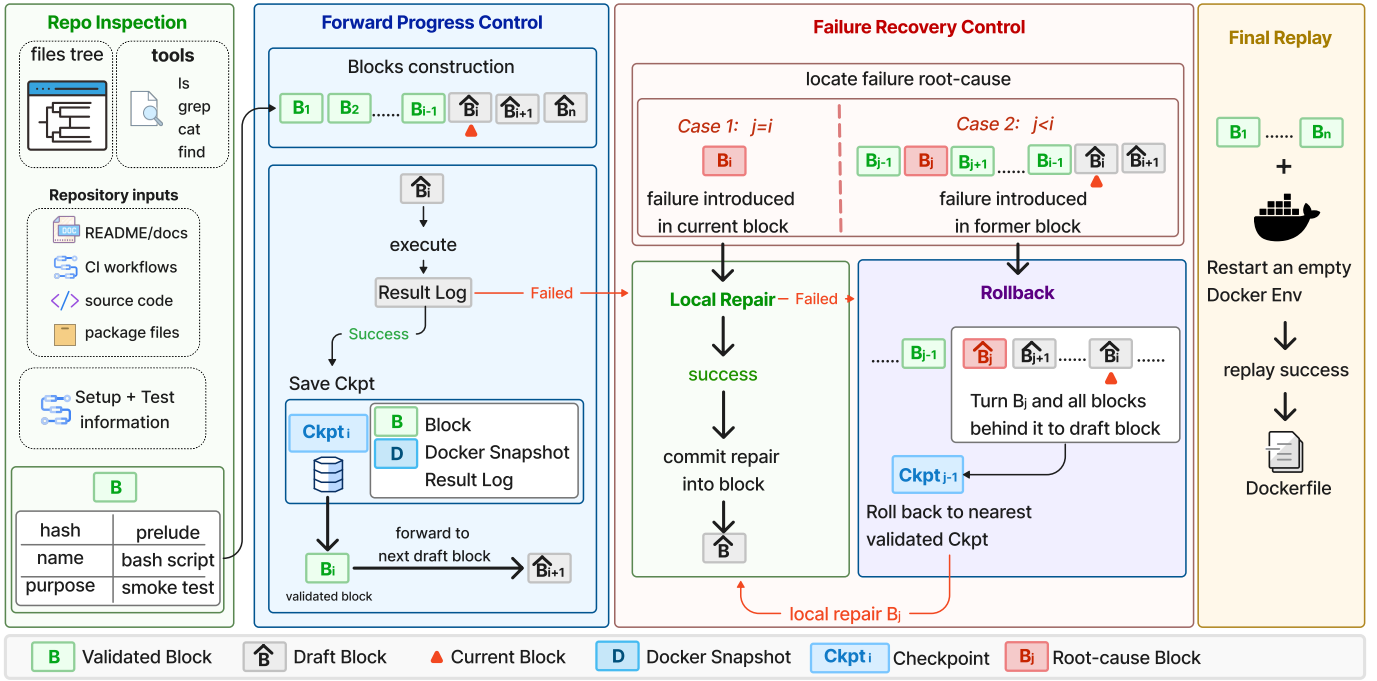


Fig. 2. Overall Approach

why.” This approach section is better than it could be on motivating the design decisions, but there are still points where things are enumerated without being explained/motivated. The motivation for read-only/non-destructive is good. But, we don’t actually know why we need these two types of evidence matters. We obviously can’t explain the whole pipeline up front, but a little bit of a preface is useful; the reader doesn’t know how it works fully yet so we have to hold their hand (this comment is much longer than the 1–2 sentences required to address it ;-)). The initialization of our framework begins with a strictly read-only inspection step designed to gather context without prematurely changing and polluting the container environment. Using only non-destructive Bash commands (e.g., `ls`, `find`, `grep`, and `cat`), the agent systematically navigates the repository to extract two critical types of information. First, it searches for *setup evidence*, includes dependency manifests (e.g., `requirements.txt`), and legacy build scripts. This is used to construct “Bash Script” part in each block. Second, it identifies *test evidence* by locating test directories and test frameworks (e.g., `pytest` and `mvn test`). This is used to construct “Validation” part in each block. By aggregating these static artifacts, this phase constructs a comprehensive knowledge base that serves as the foundational prompt context, enabling the LLM controller to accurately formulate the executable setup blocks in the subsequent phase.

CLG: Do we still do structured retrieval/a knowledge graph? It was in the ISSTA paper and it’s not here. If it’s just very simple traversal, it’s fine, but we don’t know either way. No, because the cost of KG is too high, so we cut it. We find that

explore the repo through simple command is better.

B. Block-State Construction and Forward Progress

1) *Constructing Blocks:* HerAgent adopts a global planning strategy to construct the ordered sequence of setup blocks. Initially, HerAgent takes setup evidence and test evidence as input, to synthesize blueprint of draft blocks. CLG: is this done via a single prompt? What does it contain, roughly? I don’t think we want a full prompt but we need a bit more intuition on how this is done (both at a high level and in terms of inferring block boundaries).

Block Construction Prompt:

Given a software repository mounted at `/workspace/repo` inside a fresh Docker container, generate an ordered sequence of replayable setup blocks for preparing the repository environment. Return JSON only. Split the setup into coarse-grained phases, such as preflight, system dependencies, language dependencies, and build test preparation. Each block must be executable with POSIX `sh`, deterministic, idempotent, and replayable. Avoid long-running services, interactive commands, background daemons, and host-specific paths. Validate build/test preparation using lightweight checks, such as tool-version checks, dependency-resolution checks, import checks, package-manager collection checks, or test-discovery checks, rather than running the full test suite. Do not create, edit, or monkeypatch application source files to force tests to pass. The output must include each block’s name, purpose, bash script, validation commands.

To ensure semantic consistency, the setup process is partitioned along natural functional boundaries rather than arbitrary script lengths. This typically involves separating the process into distinct, cohesive stages, such as system dependencies, language runtimes, and project dependencies. The blocks sequence is represent as

$$\mathcal{B} = [B_1, \dots, B_n]$$

Each block is represented as

$$B_i = \langle \text{Hash}, \text{Name}, \text{Purpose}, \text{Prelude}, \text{BashScript}, \mathcal{V} \rangle.$$

Every B_i has its only *Hash*, *Name* and semantic *Purpose*. The *Prelude* record the introduced changing variables, such as `APP_DIR`, `PROJECT_DIR`, `VENV_DIR`. The *setup script* field is an executable Bash body. The $\mathcal{V}$ is the validation targets associated with B_i . **CLG: I think this is the biggest hole in the approach section right now. Where does validation come from, how are commands obtained, how are they assigned to a block. There was quite a bit of content about this in ISSTA largely because it was centered on the pyramid collection pipeline, and those phases mined/assigned validation and test commands. We all agree that it's good the maturity classification scheme is gone, but the pipeline told us where $\mathcal{V}$ came from and this text doesn't. Where/how? Each block B includes a lightweight validation commands $\mathcal{V}$ that checks whether the environment state produced by B is usable. To avoid conflating environment validation with application correctness, block validations are intentionally lightweight: they verify that tools, dependencies, and build entry points are available, not that the entire test suite passes. We sanitize or rewrite heavy or brittle validations, such as replacing full `pytest` execution with `collect-only` checks.**

Blocks are created from setup and test evidence in a conservative order and semantic consistency. If an earlier validated block is modified, that block and the following checkpoint suffix are invalidated. Furthermore, cross-block shell state is made explicit: persistent changes in the *prelude* are written to a centralized `env.sh` file, which every subsequent block sources before execution.

CLG: It may be worth bringing back the BashFile template from the ISSTA paper — the conventions in teh scripts are kind of nice at showing strict mode + clean state which is pretty important. Maybe we want to be selective in which parts we bring back, but concretizing some of this stuff about `env.sh` would be nice.

2) *Constructing Checkpoint*: A checkpoint is recorded only after a block has executed successfully. Formally, after validating block B_i , HerAgent records

$$Ckpt_i = \langle B_i, D_i, R_i \rangle, \text{if } i > 0; \\ \langle D_i, \rangle, \text{if } i == 0.$$

D_i is the Docker state after executing the validated prefix $B_1, \dots, B_i$. In the implementation, this state is materialized by committing the running container to a checkpoint image

and recording the image reference with *Docker commit* and *Docker run*. R_i stores exit codes and logs for the validation commands that justify the checkpoint, so later repair and rollback decisions can use the structured evidence..

Each checkpoint $Ckpt_i$ is strictly bound to a specific block B_i , with its uniqueness guaranteed by *hash*. When the agent needs to roll back and purge a polluted state, the controller inspects both the block scripts (B_i) and their historical validation outcomes (R_i) to determine the precise checkpoint (C_i) to restore. All checkpoints also enforces a strict sequential dependency semantics: subsequent blocks are deemed trustworthy only if the exact prefix preceding them has been successfully verified. If a prior block B_i is modified, all subsequent blocks (B_{i+1} to B_n) are invalidated, and their corresponding checkpoints ($Ckpt_{i+1}$ to $Ckpt_n$) are discarded.

This checkpoint design resolves the extremes of command-level rollbacks and full resets. Command-level snapshots are overly fine-grained: because late-stage errors often stem from early misconfigurations, the LLM's ability to pinpoint root causes decays exponentially in accumulated trajectory noise. For example, upon encountering a failure at step 50, the LLM is forced to search through the trajectory noise of the preceding 49 steps just to pinpoint a faulty dependency introduced at step 12. Conversely, full resets guarantee purity but waste verified progress. By checkpointing at block boundaries, HerAgent achieves an optimal intermediate granularity—large enough to preserve meaningful progress, yet small enough to isolate failures for precise repair.

3) *Block-level Forward Progress*: Forward progress is controlled by the transition of validation state of a sequence of B , which is partitioned as

$$\mathcal{B} = [B_1, \dots, B_k, \hat{B}_{k+1}, \dots, \hat{B}_n],$$

where $B_1, \dots, B_k$ are validated and successful blocks, while $\hat{B}_{k+1}, \dots, \hat{B}_n$ remain draft blocks awaiting validation. To advance a draft block $\hat{B}_{k+1}$ into a validated state, HerAgent executes the entire block, including the *prelude*, *setup script*, and *setup script*. If execution succeeds, the system obtains the execution results R and runtime docker environment D , commits checkpoint $Ckpt_{k+1}$ and save it as a persistent file. The block is then promoted from $\hat{B}_{k+1}$ to B_{k+1} . Otherwise, the state of $\hat{B}_{k+1}$ remains unchanged and the system enters the repair phase. Successful completion is achieved when every block in $\mathcal{B}$ has been promoted to the validated state.

This mechanism provides HerAgent with a rigorous forward harness through adaptive step sizing. During routine setup stages, such as package installation or dependency resolution, the agent advances efficiently at block granularity. When failures occur, the system switches to finer command-level diagnosis within a bounded repair loop. As a result, HerAgent avoids consuming an LLM turn for every routine command while preserving sufficient locality to repair failures efficiently.

C. Failure Recovery through Repair and Rollback

When execution fails, HerAgent does not continue from the current polluted environment. Instead, it first locates where

the failure originates. If the failure is caused by the currently executing block B_i , HerAgent attempts a local repair within this block. Otherwise, it rolls back to a stable checkpoint and repairs the responsible setup block.

1) *Failure Localization*: CLG: Please add more detail here. This describes the input and the output of the localization procedure, but doesn't give us any information about mechanism. When execution fails at block B_i , HerAgent first identifies the block that most likely introduced the faulty state. The failed block is not necessarily the root-cause block: a later validation command may expose a dependency, configuration, or file-system error introduced by an earlier setup block. Therefore, before selecting a recovery action, HerAgent localizes failures using block-level provenance. Given a failure, HerAgent extracts the current failure logs and analyzes the $Ckpt_1$ to $Ckpt_{i-1}$ to identify which block B responsible for the missing or inconsistent resource.

The output of failure localization is a root-cause block B_j , which determines the recovery target: if $j = i$, HerAgent repairs the current block directly; if $j < i$, it rolls back to the $Ckpt_{j-1}$ and repairs B_j . In this way, HerAgent repairs the block that produced the faulty state rather than the later command where the symptom happens to appear.

2) *Local Repair*: Local repair is triggered after locating the root-cause block B_j . The repair procedure follows a four-step loop:

diagnose $\rightarrow$ trial $\rightarrow$ patch back $\rightarrow$ replay.

Before attempting any modification, HerAgent diagnoses the failure of this root-cause block. Reproducible errors, such as missing commands, modules, files, permissions, dependencies, or classpaths, proceed directly to repair. Transient errors, such as timeouts or temporary network failures, are retried once. This allows HerAgent to distinguish the block where the failure appears from the block that should have produced the missing tool, package, file, or configuration.

Then the agent proposes and executes a small number of candidate repair commands scoped to the localized block. Typical repairs include appending missing setup steps, correcting package versions or command-line flags or updating environment variables. Next, HerAgent evaluates the candidate repairs. If a candidate succeeds and handle the failure, the candidate is treated as a patch and merged into the target block, instead of being left as an interactive mutation in the live environment. CLG: OK the patch-back thing is pretty important and it's totally buried right now (and I'm about to have to quit so I can't fix it). This kind of explains why the script-centric approach, but in the middle of a paragraph in the middle of the section; might be better to be explicit about that motivation at the top of the section in overviewing the approach (IIRC the ISSTA paper did a clearer job of this). Maybe more importantly, we have a whole RQ about the patch-back mechanism ,so it needs to not be buried like it is now; should be more explicitly important as part of the methodology.

Finally, the modified block B_j is replayed from the $Ckpt_{j-1}$. A repair is accepted only if the patched block B_j succeeds under replay. This acceptance rule is stricter than simply observing that a command works once: the successful action must be incorporated into the durable setup artifact and must reproduce correctly from a clean checkpoint. If local repair fails under the given limitation, the block information, as well as the trajectory and result of the local repair, will be used to re-locate the error. CLG: ...forever? Is more information provided? Is there a bound?

3) *Checkpoint Rollback*: Rollback is not the default response to every failure. It is triggered only when the current environment is classified as polluted. CLG: when/how does this happen? This may not be the right place to answer that, but it is the place that the thought occurred to me. and local repair cannot resolve the failure. If the failure is caused by a later validation command but the evidence points to an earlier dependency block, HerAgent repairs that earlier block and invalidates it. Specifically, if failure localization identifies an earlier root-cause block B_j , HerAgent marks B_j and all subsequent blocks as invalidated $\hat{B}_j, \hat{B}_{j+1}, \dots$. It then restores the checkpoint $Ckpt_{j-1}$, prunes all checkpoints after $Ckpt_{j-1}$, and resumes local repair, execution, and validation from $\hat{B}_j$.

This checkpoint design fundamentally resolves the extremes of command-level rollbacks and full resets. Command-level rollback is too fine-grained and assumes that the latest command caused the fault, which fails when later errors come from earlier misconfigurations. Consequently, command-level rollbacks force the agent to navigate through accumulated trajectory noise, exponentially degrading its ability to pinpoint the true root cause. Full reset restores a clean environment but discards validated progress. HerAgent instead performs bounded recovery: each rollback removes the suspected faulty suffix while preserving the largest validated prefix whose block scripts, Docker state, and validation evidence remain consistent.

If a setup trajectory passed during interactive exploration but fails during final replay, HerAgent reports a reproducibility failure and attributes it to the corresponding block or validation phase. This final check ensures that the output is not a transcript of commands that happened to work in one container, but a reusable setup artifact that reconstructs the repository environment from a clean state.

D. Final Clean Replay

Final replay closes the setup loop. After all blocks are executed and validated during interactive exploration, HerAgent treats the environment as a candidate success state. However, the final output is not the temporary shell state or the explored Docker container; it is a reusable setup artifact, including the generated Dockerfile and Bash scripts.

To verify this artifact, HerAgent discards the working container state, starts a fresh Docker environment, checks out the repository at the same revision, writes the final block sequence, and executes all blocks in order without LLM intervention.

If a trajectory succeeds during interactive exploration but fails during final replay, HerAgent reports a reproducibility failure and attributes it to the corresponding block or validation phase. Thus, final replay ensures that the delivered artifact can reconstruct the repository environment from a clean state.

IV. EXPERIMENTS

““latex ““

We evaluate HerAgent with three goals. First, we test whether block-level harness improves end-to-end environment setup compared with prior environment-setup agents. Second, we isolate which components of HerAgent are responsible for the improvement. Third, we analyze failed and successful trajectories to understand when block-level checkpoints help and when they are insufficient.

A. Research Questions

RQ1: Overall effectiveness. How effective is HerAgent compared with prior environment-setup agents on existing benchmarks?

RQ2: Contribution of progress-control components. Which components of HerAgent contribute to its effectiveness?

RQ3: Behavior under failure and recovery. How does HerAgent behave when setup fails during long-horizon environment construction?

B. Datasets and Baselines

ExecutionAgent. The ExecutionAgent-Bench include 50 repositories with 14 language, and the dataset evaluates the ability to build arbitrary multilingual repositories and run their test suites from CI workflows. Our baseline is ExecutionAgent. Both HerAgent and the ExecutionAgent baseline are evaluated using the same base model ([xxx]). The ExecutionAgent artifact does not provide exact commits for all repositories. For repositories without released commits, we fix a snapshot for each repository and rerun both ExecutionAgent and HerAgent under the same commit setting.

SetupBench. SetupBench emphasizes a environment-bootstrap skill: starting from a bare Linux sandbox, an agent must install packages, resolve dependency conflicts, initialize databases, and configure background services. It includes 93 instances and these tasks span seven language ecosystems, five database engines, and multi-service orchestration scenarios, each accompanied by a natural language problem statement and a deterministic success command. For our evaluation, the primary baseline is OpenHands, which is the main agent framework evaluated in the original benchmark. Under our matched-model control, we compare HerAgent directly against OpenHands using the same [xxx] base model.

Repo2Run. The Repo2Run dataset focuses on generating reproducible Docker environments specifically for Python repositories. Tasks are validated by successfully building the Docker image and executing the repository’s unit tests. For cost control, we evaluate on a fixed subset of [xxx] repositories selected prior to running any experiments; the exact repository

URLs, commit hashes, validation commands, and exclusion reasons are released to avoid selection bias. We adopt the original Repo2Run agent as our primary baseline and executed with the same [xxx] base model.

C. Methodology of RQ1

D. Methodology of RQ2

E. Methodology of RQ3

F. Metrics

We use four groups of metrics.

Effectiveness. We report setup success and validation success. Setup success means that the generated artifact can install the required system, runtime, and project dependencies without fatal error. Validation success means that the dataset-specific validation command passes after replay.

Reproducibility. We report final artifact replay success. A run is reproducible only if the final Dockerfile, BashFile sequence, or setup script can reconstruct the environment from a clean container and pass the final validation command.

Efficiency. We report number of input/output tokens, execution commands and number of sandbox interactions. These metrics test whether HerAgent reduces wasted exploration rather than merely improving success through more trial and error.

Progress-control behavior. For HerAgent and its ablations, we report the number of generated blocks, committed checkpoints, local repair attempts, successful patch-back operations, checkpoint rollbacks, replay attempts, and validation regressions. A validation regression occurs when a previously passed validation target later fails after a repair or rollback.

V. RESULTS

A. RQ1: Overall Effectiveness

Table ?? reports the main results. HerAgent achieves higher validation success than the strongest matched baseline on all three datasets: [xxx] vs. [xxx] on ExecutionAgent, [xxx] vs. [xxx] on SetupBench, and [xxx] vs. [xxx] on Repo2Run. The replay-success results show the same trend, indicating that HerAgent does not merely find transient interactive states but produces artifacts that can be reconstructed from a clean environment.

The improvement is most visible on tasks that require multiple dependent setup stages. In these cases, a single failed command often reflects an earlier runtime, dependency, or path decision. Command-level agents must infer this relation from a long noisy trajectory, while whole-script methods often regenerate or revise a large artifact. HerAgent instead records progress at semantic block boundaries and can restore the nearest validated checkpoint before repairing the suspected root-cause block.

Efficiency results are shown in Table ?. HerAgent uses [xxx]% fewer LLM calls and [xxx]% fewer sandbox interactions than the command-level baseline on ExecutionAgent. On Repo2Run, HerAgent uses [xxx] more/less wall-clock time than Repo2Run, mainly because checkpoint creation and

TABLE I
RQ1: END-TO-END EFFECTIVENESS AND EFFICIENCY UNDER MATCHED-MODEL SETTINGS.

Dataset	Method	Success Rate(%)	Average Cost(\$)	Average Time(min)	Average Steps
ExecutionAgent-bench	ExecutionAgent(<i>gpt-4o-mini</i>)	66.0	0.16	74	-
	AutoGPT(<i>gpt-4o-mini</i>)	8.0	-	-	-
	HerAgent (<i>gpt-4o-mini</i>)	[xxx]	-	-	-
	HerAgent (<i>gpt-5.4</i>)	[xxx]	[xxx]	[xxx]	[xxx]
Repo2Run-bench	Repo2Run(<i>gpt-4o</i>)	86.0	3.56	29.0	18
	SWE-agent(<i>gpt-4o</i>)	9.0	-	-	-
	HerAgent (<i>gpt-4o</i>)	[xxx]	-	-	-
	HerAgent (<i>gpt-5.4</i>)	[xxx]	0.18	[xxx]	[xxx]
SetupBench (Repo Setup subset)	OpenHands(<i>gpt-4.1</i>)	46.3	2.2	-	29.5
	OpenHands(<i>Claude 4 Sonnet</i>)	57.4	10.2	-	47.1
	HerAgent (<i>gpt-4.1</i>)	[xxx]	-	-	-
	HerAgent (<i>gpt-5.4</i>)	83.3	0.35	6.93	29.9

TABLE II
RQ2: ABLATION STUDY OF HERAGENT PROGRESS-CONTROL COMPONENTS. A CHECKMARK INDICATES THAT THE CORRESPONDING HERAGENT COMPONENT IS ENABLED; “-” INDICATES THAT THE COMPONENT IS REMOVED OR REPLACED BY THE ABLATED STRATEGY.

Control setting	Ablated dimension	Block forward	Block recovery	Local repair	Ckpt. rollback	Clean replay	Val. succ.	Replay succ.	Median tokens	Steps
Full HerAgent		✓	✓	✓	✓	✓	[xxx]	[xxx]	[xxx]	[xxx]
Whole-script	Forward	-	-	-	-	✓	[xxx]	[xxx]	[xxx]	[xxx]
	Recovery	✓	-	-	-	✓	[xxx]	[xxx]	[xxx]	[xxx]
	Forward + Recovery	-	-	-	-	✓	[xxx]	[xxx]	[xxx]	[xxx]
Single-command	Forward	-	✓	✓	✓	✓	[xxx]	[xxx]	[xxx]	[xxx]
	Recovery	✓	-	✓	-	✓	[xxx]	[xxx]	[xxx]	[xxx]
	Forward + Recovery	-	-	✓	-	✓	[xxx]	[xxx]	[xxx]	[xxx]
Component removal	w/o local repair	✓	✓	-	✓	✓	[xxx]	[xxx]	[xxx]	[xxx]
	w/o checkpoint rollback	✓	✓	✓	-	✓	[xxx]	[xxx]	[xxx]	[xxx]
	w/o final clean replay	✓	✓	✓	✓	-	[xxx]	[xxx]	[xxx]	[xxx]

replay introduce overhead. However, this overhead is offset by a higher final replay-success rate, which is the primary target for reusable environment construction.

Answer to RQ1. HerAgent improves end-to-end setup and replay success across the three benchmark settings. The gain is not only a higher interactive success rate; the replay results show that HerAgent more often produces a durable setup artifact.

B. RQ2: Contribution of Progress-Control Components

Table II shows the ablation results. Removing block-level forward execution reduces validation success from [xxx] to [xxx], showing that batching routine commands into semantic blocks is important for reducing unnecessary decisions. Removing local repair with patch-back reduces replay success from [xxx] to [xxx], showing that successful interactive fixes must be written back into the durable artifact. Removing checkpoint rollback causes more failures on tasks with polluted states or early runtime mistakes. Removing final replay

increases apparent success during exploration but reduces true reproducibility.

The strongest degradation appears in the variants that remove checkpoint rollback and patch-back. This supports the central claim of the paper: the key problem is not simply generating better commands, but controlling which progress should be preserved, which state should be discarded, and which repair should become part of the final artifact.

Answer to RQ2. All four components contribute to HerAgent, but checkpoint rollback and patch-back are the most directly tied to reproducible success. Block-level forward execution mainly improves efficiency, while final replay prevents false positives caused by hidden interactive state.

C. RQ3: Failure and Recovery Behavior

Table ?? summarizes HerAgent’s internal progress-control behavior. Across [xxx] runs, HerAgent commits [xxx] checkpoints, performs [xxx] local repair attempts, and restores checkpoints [xxx] times. Among failed runs, [xxx]% involve failures that are exposed later than the block that introduced

the faulty state. These cases are exactly where command-level rollback is least reliable.

Manual inspection shows three common success patterns. First, HerAgent recovers from runtime-version errors by rolling back to the runtime block rather than repairing the later test command. Second, HerAgent avoids keeping dirty interactive mutations by accepting a repair only after patch-back and replay. Third, HerAgent preserves expensive earlier progress, such as system dependency installation, while discarding only the invalidated suffix.

We also observe three common failure patterns. First, some repositories require external services, credentials, or network resources that are not available in the sandbox. Second, some validation commands are flaky or underspecified, making it hard to distinguish environment failure from project-level test nondeterminism. Third, some failures require repository-specific domain knowledge that cannot be inferred from static artifacts or logs alone.

Answer to RQ3. HerAgent is most useful when failures are caused by earlier setup decisions but exposed later by build, test, or run commands. Its main limitation is not ordinary dependency repair, but missing external resources, ambiguous validation targets, and failures that cannot be localized from repository evidence.

VI. DISCUSSION

The results suggest that environment setup should not be treated as either a sequence of independent commands or a single script generation problem. Real setup trajectories contain routine progress, late-exposed failures, hidden state pollution, and repairs that must become reproducible artifacts. HerAgent addresses this by separating three notions that are often mixed in prior agents: the current live container, the validated setup prefix, and the final replayable artifact.

The ablation results also clarify the role of block-level design. Blocks are not only a token-saving device. They define recovery boundaries. Without such boundaries, a method may either roll back too locally and keep polluted state, or reset too globally and discard useful progress. The checkpointed block sequence gives the controller a middle granularity: large enough to make progress, but small enough to repair.

VII. THREATS TO VALIDITY

Benchmark coverage. The three datasets cover multilingual test execution, environment bootstrapping, and Docker-based Python setup, but they do not cover all possible repository environments. In particular, projects requiring private credentials, paid services, GPUs, or complex multi-node deployment may behave differently.

Baseline reproducibility. Some baselines may not expose all prompts, model versions, repository commits, or validation commands. We mitigate this by using matched-model comparisons whenever possible and by separating original-paper reference results from our primary claims.

Validation target quality. A passed validation command does not prove that the environment supports every user workflow. We therefore interpret success as passing the dataset-specific readiness target, not as complete repository usability. We release all validation commands so that future work can extend or challenge them.

LLM variance. Environment setup agents can be sensitive to decoding variance. We use fixed prompts, fixed model settings, and [xxx] repeated runs where cost allows. For expensive settings, we report exact run logs and avoid overclaiming statistical significance.

Infrastructure variance. Network availability, package mirrors, and external dependency versions may change over time. We record timestamps, base images, package-manager logs, repository commits, and generated artifacts to support reproduction.

VIII. RELATED WORK

A. Environment Setup and Synthesis

Automated environment setup has recently become an important problem for repository-level software engineering agents. Unlike isolated code generation tasks, repository setup requires an agent to recover the executable context of an existing project, including system packages, language runtimes, project dependencies, build artifacts, services, and validation commands. SetupBench [1], EnvBench [5], Repo2Run [3], and RATBench [8] evaluate whether agents can transform raw repositories into executable states. ResearchEnvBench [9], CSR-Bench [10], and Multi-Docker-Eval [11] further show that research-code deployment, multilingual repositories, and Docker-based setup expose dependency, runtime-version, and service-configuration failures that are not captured by ordinary code-generation benchmarks.

Prior setup systems differ in how they represent and control the setup process. ExecutionAgent [2] follows an interactive execution paradigm: the agent executes commands in a sandbox, observes failures, and iteratively revises its decisions until it can run project tests. Repo2Run [3] targets reproducible executable environments and introduces rollback during Dockerfile construction. Other systems emphasize artifact generation: PiPer [4] generates setup scripts, Installmatic [7] synthesizes Dockerfiles from installation instructions, and Vibenix [12] explores AI-assisted packaging with Nix. These approaches reveal a granularity tradeoff: command-level exploration is flexible but can leave useful progress implicit in the container state, while whole-artifact generation is replayable but coarse for local repair.

Recent concurrent work expands environment setup into a broader agent capability. MEnvAgent [6] studies scalable polyglot environment construction and reuse, EvoConfig [13] uses self-evolving multi-agent diagnosis, DockSmith [14] builds reliable Docker environments with loop detection and cross-task memory, and SetupX [15] studies learning from past setup failures with Docker snapshot rollback. Large-scale environment synthesis efforts such as SWE-Factory [16], daVinci-Env/OpenSWE [17], and SWE-Universe [18] further

show that constructing verifiable environments has become infrastructure for training and evaluating software engineering agents at scale. These works primarily address multi-repository scale, memory, or data generation, while HerAgent focuses on progress control inside one difficult repository setup trajectory.

B. Executable Validation and Reproducibility

Executable environments are also central to software engineering agents and their validation loops. SWE-bench [19], SWE-agent [20], OpenHands [21], and CodeAct [22] all rely on command execution and test feedback to ground agent behavior. EnvGraph [23] connects executable repository-level code generation with environment alignment, and CI-oriented studies such as Beyond Software Development [24] and SWE-CI [25] emphasize long-horizon build, test, and regression validation. These systems usually treat the environment as an execution substrate, whereas HerAgent targets the earlier prerequisite: constructing a reproducible environment that can support tests, CI-style checks, service launch, and user-facing commands.

Finally, HerAgent builds on the long-standing goal of reproducible environments. Docker [26], Nix [27], and ReproZip [28] show that dependencies, provenance, and execution state must be made explicit to reproduce software reliably. SWE-World [29] explores a contrasting direction by replacing physical Docker execution with a learned surrogate to reduce environment cost. HerAgent instead assumes real execution is necessary for user-ready validation, because dependency conflicts, service failures, hidden state pollution, and repository-specific runtime behavior often appear only in a physical environment. Our block-level checkpoints therefore combine a replayable BashFile state, a Docker snapshot, and oracle results, giving the agent a recovery boundary that is larger than a single command but smaller than a full regenerated setup script.

REFERENCES

- [1] A. Arora, J. Jang, and R. Z. Moghaddam, "Setupbench: Assessing software engineering agents' ability to bootstrap development environments," *arXiv preprint arXiv:2507.09063*, 2025.
- [2] I. Bouzenia and M. Pradel, "You name it, i run it: An llm agent to execute tests of arbitrary projects," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1054–1076, 2025.
- [3] R. Hu, C. Peng, X. Wang, J. Xu, and C. Gao, "Repo2run: Automated building executable environment for code repository at scale," *arXiv preprint arXiv:2502.13681*, 2025.
- [4] A. Kovrigin, A. Eliseeva, K. Grotov, E. Bogomolov, and Y. Zharov, "Piper: On-device environment setup via online reinforcement learning," *arXiv preprint arXiv:2509.25455*, 2025.
- [5] A. Eliseeva, A. Kovrigin, I. Kholkin, E. Bogomolov, and Y. Zharov, "Envbench: A benchmark for automated environment setup," *arXiv preprint arXiv:2503.14443*, 2025.
- [6] C. Guo, J. Wu, S. He, Y. Chen, Z. Kuang, S. Fan, B. Chen, S. Bao, J. Liu, H. Wu, Q. Zhu, W. Che, and H. Wang, "Menvagent: Scalable polyglot environment construction for verifiable software engineering," *arXiv preprint arXiv:2601.22859*, 2026.
- [7] L. Milliken, S. Kang, and S. Yoo, "Beyond pip install: Evaluating llm agents for the automated installation of python projects," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2025, pp. 1–11.
- [8] R. Huang, D. Hua, Y. Sun, S. Ding, H. Yuan, D. Wang, and Y. Yang, "Rat: Runanything via fully automated environment configuration," *arXiv preprint arXiv:2604.23190*, 2026.
- [9] Y. Wang, C. Zhang, B. Chen, Z. Huai, Z. Dai, X. Chen, Y. Wang, Y. Zheng, J. Gong, and X. Qiu, "Researchenvbench: Benchmarking agents on environment synthesis for research code execution," *arXiv preprint arXiv:2603.06739*, 2026.
- [10] Y. Xiao, R. Wang, L. Kong, D. Golac, and W. Wang, "CSR-bench: Benchmarking LLM agents in deployment of computer science research repositories," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2025, pp. 12 705–12 723.
- [11] K. Fu, T. Liu, Z. Shang, Y. Ma, J. Yang, J. Liu, and K. Bian, "Multi-docker-eval: A shovel of the gold rush benchmark on automatic environment building for software engineering," *arXiv preprint arXiv:2512.06915*, 2025.
- [12] M. Schwaighofer, M. Monis, N. Saavedra, J. F. Ferreira, and R. Mayrhofer, "Vibenix: An ai assistant for software packaging with nix."
- [13] X. Guo, J. Kuang, L. Pan, Y. Li, Y. Li, H.-T. Zheng, Y. Shen, D. Yin, and X. Sun, "Evoconfig: Self-evolving multi-agent systems for efficient autonomous environment configuration," *arXiv preprint arXiv:2601.16489*, 2026.
- [14] J. Zhang, L. Ma, Y. Li, F. Wan, D. Qi, X. Zhao, J. Hou, Z. Xie, M. Ren, X. Wu, Z. Huang, L. Chen, Y. Ma, Q. Han, and X. Zhang, "Docksmith: Scaling reliable coding environments via an agentic docker builder," *arXiv preprint arXiv:2602.00592*, 2026.
- [15] Z. Zhou, Z. Ren, Y. Wu, Y. Xiong, W. Zhou, C. Peng, D. Zhang, B. Yan, X. Zhou, and F. Wu, "Setupx: Can llm agents learn from past failures in functionality-correct code repository setup?" *arXiv preprint arXiv:2605.26186*, 2026.
- [16] L. Guo, Y. Wang, C. Li, P. Yang, J. Chen, W. Tao, Y. Zou, D. Tang, and Z. Zheng, "Swe-factory: Your automated factory for issue resolution training data and evaluation benchmarks," *arXiv preprint arXiv:2506.10954*, 2025.
- [17] D. Fu, S. Wu, Y. Wu, Z. Peng, Y. Huang, J. Sun, J. Zeng, M. Jiang, L. Zhang, Y. Li, J. Hu, L. Liu, J. Hou, and P. Liu, "davinci-env: Open swe environment synthesis at scale," *arXiv preprint arXiv:2603.13023*, 2026.
- [18] M. Chen, L. Zhang, Y. Feng, X. Wang, W. Zhao, R. Cao, J. Yang, J. Chen, M. Li, Z. Ma, H. Ge, Z. Zhang, Z. Cui, D. Liu, J. Zhou, J. Sun, J. Lin, and B. Hui, "Swe-universe: Scale real-world verifiable environments to millions," *arXiv preprint arXiv:2602.02361*, 2026.
- [19] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQM66>
- [20] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, 2024.
- [21] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "Openhands: An open platform for ai software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.
- [22] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better llm agents," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [23] R. Pan, J. Shen, L. Wu, Y. Zhu, Z. Yang, Y. Zhang, L. Zhang, and H. Zhang, "Toward executable repository-level code generation via environment alignment," *arXiv preprint arXiv:2604.03622*, 2026.
- [24] J. Wang, L. Applis, and A. Roychoudhury, "Beyond software development: Continuous integration with coding agents," OpenReview submission to ICLR 2026, 2026.
- [25] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao, "Swe-ci: Evaluating agent capabilities in maintaining codebases via continuous integration," *arXiv preprint arXiv:2603.03823*, 2026.
- [26] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.
- [27] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A safe and policy-free system for software deployment," in *Proceedings of the 18th Large Installation System Administration Conference*, 2004, pp. 79–92.
- [28] F. Chirigati, R. Rampin, D. Shasha, and J. Freire, "Reprozip: Computational reproducibility with ease," in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 2085–2088.

- [29] S. Sun, H. Song, L. Huang, J. Jiang, R. Le, Z. Lv, Z. Chen, Y. Hu, W. Luo, W. X. Zhao, Y. Song, H. Xu, T. Zhang, and J.-R. Wen, “Swe-world: Building software engineering agents in docker-free environments,” *arXiv preprint arXiv:2602.03419*, 2026.